Machine Learning Fundamentals

(Sparse) Linear Models

ETSIINF
Universidad Politécnica de Madrid
b.mihaljevic@upm.es



Table of Contents

1. Representation
2. Learning
3. Interpretation
4. Sparsity
5. Generalizations

Linear model: first example

Suppose we predict a risk score from age, smoking, and exercise:

$$\widehat{\text{risk}} = \beta_0 + \beta_{\text{age}}\text{age} + \beta_{\text{smoker}}\mathbb{I}(\text{smoker}) + \beta_{\text{exercise}}\text{exercise}.$$

Reading the coefficients

- β_0: baseline prediction for the reference instance.
- Positive coefficients push the prediction up; negative coefficients push it down.
- A coefficient near zero means little direct contribution **in this fitted model**.
- Every coefficient is a partial effect: the other features stay fixed.

Why linear models?

- In low dimensions they can be directly interpretable.
- In high dimensions, simple linear models can be very competitive.
- A non-linear representation can still use a linear model after transformation.
- Linear models are often used as local or global surrogates for black-box models.
- In high dimensions, interpretation becomes fragile; sparsity helps, but does not solve everything.

Regression

Model the dependence of a regression target Y on features $X_1, \ldots, X_n$,

$$Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n + \epsilon,$$

where

- β_0 is the intercept (not multiplied with a feature);
- $\beta_j, j \neq 0$ are the coefficients;
- ϵ is the error (random variable).

Interpretability

The coefficients $\beta_j, j \neq 0$ quantify the *partial* influence of the different features.

Classification

The regression model, as such, is not appropriate for classification. For binary classification, the logistic regression model is,

$$\log\left(\frac{P(Y=1)}{P(Y=0)}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n.$$

Interpretability

It is harder to interpret the coefficients $\beta_j, j \neq 0$ since their relationship to class probability is non-linear.

Table of Contents

1 Representation

2 Learning

3 Interpretation

4 Sparsity

5 Generalizations

Learning (aka fitting)

- The "standard" way: fit linear and logistic regression by maximum likelihood.
- Alternative: use penalized likelihood to, e.g., enforce sparsity.

Linear regression - ordinary least squares

Fit β_j by minimizing the squared differences between the actual and the predicted outcomes:

$$\hat{\boldsymbol{\beta}} = \underset{\beta_0,\ldots,\beta_n}{\arg\min} \sum_{i=1}^{N} \left(y^{(i)} - \left(\beta_0 + \sum_{j=1}^{n} \beta_j x_j^{(i)} \right) \right)^2 ;$$

the unique solution, if $\mathbf{X}$ has full rank, is $\hat{\beta} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$, where $\mathbf{X}$ is the $N \times (n+1)$ design matrix with a leading column of ones.

Design matrix

R makes the design matrix explicit with `model.matrix()`.

```
df <- data.frame(
  age = c(25, 40, 55),
  smoker = factor(c("no", "yes", "no")),
  program = factor(c("A", "B", "C"))
)
data.frame(model.matrix(~ age + smoker + program, df), check.names =
```

```
##   (Intercept) age smokeryes programB programC
## 1           1  25         0        0        0
## 2           1  40         1        1        0
## 3           1  55         0        0        1
```

Notes

- The first column is the intercept: a column of ones.
- The matrix is not centered unless we explicitly center or standardize inputs.
- Categorical features become columns in the design matrix.

Learning (aka fitting)

In addition, there are alternative formulations that lead to a linear model.

Alternative formulations

- Linear discriminant analysis (classification)
- Linear support vector machine
- Categorical naive Bayes (classification)

Interpretation

Not much different from what we are going to discuss.

Table of Contents

1 Representation

2 Learning

3 Interpretation

4 Sparsity

5 Generalizations

Linear regression

Interpreting $\beta_j, j \neq 0$, numerical feature

- Increasing X_j by one increases Y by β_j.
- **Partial effect**: the **remaining features are kept constant** while X_j is changed.

Interpreting β_0

- For an instance with all numerical feature values at zero and the categorical feature values at the reference categories, the model prediction is the intercept weight.
- Note, however, that having all values at zero may not make sense in some cases.

Categorical features

So far X_j was a real-valued variable. Categorical variables are handled by expanding the design matrix.

A feature with K categories is encoded as $K-1$ binary features, with 1 representing the presence of that category (one-hot encoding). One category is the reference category.

```
x=factor(c('A', 'A', 'B', 'B', 'C'))
y = 1:5
lm(y ~ x)$coefficients
```

```
## (Intercept)          xB          xC
##         1.5         2.0         3.5
```

K-1 columns

- Having K columns would give perfect multicollinearity: the sum of all columns equals the intercept.
- With perfect multicollinearity the solution $\hat{\beta}_j$ is no longer unique.

Categorical features

```
x=factor(c('A', 'A', 'B', 'B', 'C'))
y = 1:5
lm(y ~ x)$coefficients
```

```
## (Intercept)          xB          xC
##         1.5         2.0         3.5
```

Interpreting $\beta_j, j \neq 0$, categorical feature

- Instead of β_j, we have β_{jk}, where k is the category index, $k = 1, \ldots, K-1$.
- Changing from the reference category to category l increases Y by β_{jl}.
- Changing the category from l to k sets the binary variable X_l to 0 and X_k-th to 1. The effect of this change on the outcome is $\beta_{jk} - \beta_{jl}$.
- **Partial effect**: the **remaining features are kept constant** while X_j is changed.

Categorical features

Too many categories

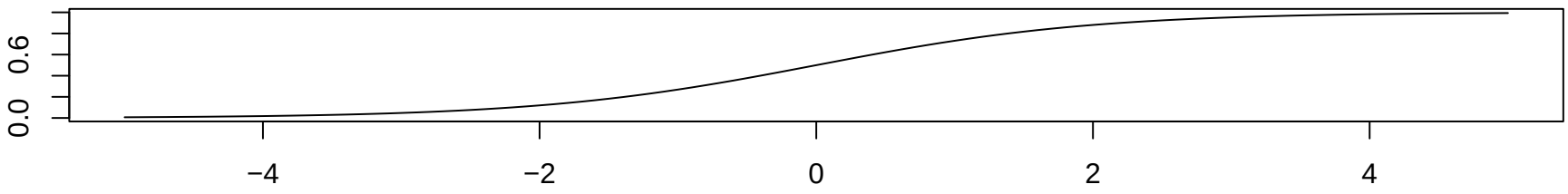
- One categorical feature with K levels adds $K - 1$ coefficients.
- Rare levels have few observations: estimates can be noisy or undefined.
- Many one-hot columns increase overfitting risk and reduce interpretability.
- Possible responses: combine rare categories, use regularization, or use a carefully validated encoding.

Logistic regression

As mentioned, the interpretation of β_js is not as straightforward as in linear regression, as their relation to class probability is not linear. The logistic regression model, for binary classification,

$$\log\left(\frac{P(Y=1)}{P(Y=0)}\right) = \eta = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n,$$

uses the logistic function to squeeze the output of the linear model between 0 and 1, $\text{logistic}(\eta) = \frac{1}{1+\exp(-\eta)}$.



Logistic regression

The logistic regression is a linear model for the *log odds*.

$$\log\left(\frac{P(Y=1)}{P(Y=0)}\right) = \log\left(\frac{P(Y=1)}{1-P(Y=1)}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n.$$

Odds

- The probability of event divided by probability of no event, $\frac{P(Y=1)}{P(Y=0)}$.
- For example, $\frac{P(Y=1)}{P(Y=0)} = 2$ means $Y=1$ is $2\times$ as likely as $Y=0$.

Logistic regression

$$\log\left(\frac{P(Y=1)}{P(Y=0)}\right) = \log\left(\frac{P(Y=1)}{1-P(Y=1)}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n.$$

Interpretation, numerical feature

- Increasing X_j by 1, keeping all other variables constant, increases the log odds by β_j.
- The odds get multiplied by $\exp(\beta_j)$.
- For example, for $\beta_j = 0.7$, increasing X_j by one multiplies the odds by $\exp 0.7 \approx 2$; how much $P(Y=1)$ goes up depends on the original odds.
- The β_j alone does not give us the absolute increase in $P(Y=1)$ when X_j increased by 1 (keeping all other variables constant).

Logistic regression

Odds are not probabilities

Multiplying the odds by 2 does not double the probability.

old $P(Y=1)$	old odds	new odds	new $P(Y=1)$
0.20	0.25	0.50	0.33
0.50	1.00	2.00	0.67
0.80	4.00	8.00	0.89
0.95	19.00	38.00	0.97

At very low or very high probabilities, changing the linear score can have a small effect on the probability.

Logistic regression

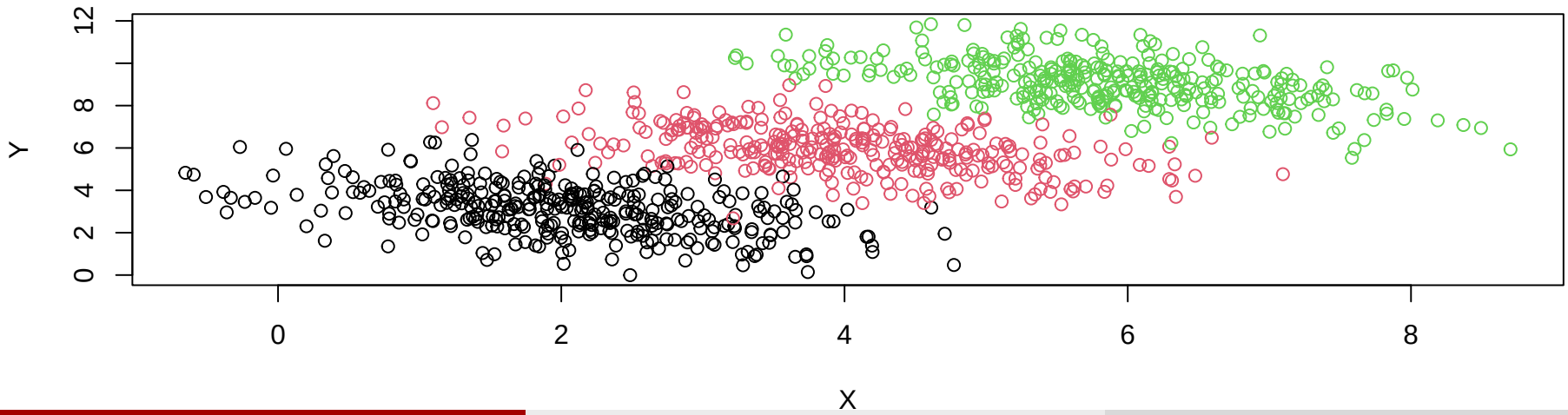
Multiclass

- Binary logistic regression has one coefficient vector: class 1 versus class 0.
- Multiclass logistic regression has several coefficient vectors.
- Each coefficient is interpreted relative to a class comparison.
- Interpretability scales poorly with many classes and many features.

Partial effect

As mentioned β_j measures the effect of the variable when all other variables in the model are kept constant.

```
n <- 1000
Z <- sample(c(1,2,3), size=n, replace=TRUE)
X <- 2*Z + rnorm(n)
Y <- 4*Z - 0.5*X + rnorm(n)
plot(X, Y, col = Z)
```



Partial effect

- β_j does not measure the total effect of X_j on Y unless X_j is independent of other features.
- Each β_j represents the additional (direct) effect of adding X_j to the model, having accounted for the effects of all other variables.
- Adding or removing variables from the model, and retraining, will change other coefficients (unless removed variables independent of others).

Coefficients and importance

Common trap

Does a small coefficient mean that a variable is irrelevant?

Coefficients and importance

Common trap

Does a small coefficient mean that a variable is irrelevant?

Not necessarily.

- The coefficient depends on scale and preprocessing.
- It is conditional on the other variables in the model.
- Correlated variables can share or hide each other's contribution.
- Regularization can shrink coefficients even when predictors are useful.
- A coefficient is a statement about a fitted model, not a universal property of the variable.

Feature importance

Relative feature importance: comparing coefficients of different predictors.

```
n <- 1e3
x1 <- rnorm(n)
x2 <- rnorm(n, sd = 40)
y <- x1 + x2 + rnorm(n)
lm(y ~ x1 + x2)$coefficients
```

```
## (Intercept)          x1          x2
## -0.02111297  0.97188617  1.00024670
```

However, do X_1 and X_2 have a comparable effect on Y?

Feature importance

Relative feature importance: comparing coefficients of different predictors.

```
n <- 1e3
x1 <- rnorm(n)
x2 <- rnorm(n, sd = 40)
y <- x1 + x2 + rnorm(n)
lm(y ~ x1 + x2)$coefficients
```

```
## (Intercept)          x1          x2
## -0.02111297  0.97188617  1.00024670
```

However, do X_1 and X_2 have a comparable effect on Y?

```
lm(y ~ scale(x1) + scale(x2))$coefficients
```

```
## (Intercept)   scale(x1)   scale(x2)
##   1.1688219   0.9462589  42.0957900
```

Feature importance

```
lm(y ~ scale(x1) + scale(x2))$coefficients
```

```
## (Intercept)   scale(x1)   scale(x2)
##   1.1688219   0.9462589  42.0957900
```

Now coefficients do give relative feature importance, as they take into account the features' variances.

Feature importance measures:

- The coefficients after having scaled the data.
- The coefficients fit with unscaled data, multiplied by the standard deviation of the corresponding feature.

Standardization

Each feature: Subtract the mean and divide by standard deviation, $\frac{x-\hat{x}}{s_x}$.

```
df <- data.frame(age=c(18,21,30), gender=c(0, 0, 1))
scale(df)[,]
```

```
##                age     gender
## [1,] -0.8006408 -0.5773503
## [2,] -0.3202563 -0.5773503
## [3,]  1.1208971  1.1547005
```

- Note that binary predictors (gender) are typically not scaled.
- Subtracting the mean, though common, is not strictly necessary.

Standardization

Now the reference instance is more typical: all predictors at their mean.

- This still may not be a viable instance, but it is very likely to be more meaningful than setting all features to 0.
- Now we can interpret the intercept as the expected value of Y when the predictors are set to their means.

However, after standardization, the units have changed. For example

- a non-negative variable like age will have negative values.
- a binary variable will no longer be encoded as 0/1.

Interpretation

- Increasing one unit, now, means increasing by one standard deviation.

Standardization

Pros

- Useful for comparing coefficients of different predictors within a model.
- Although: How to compare coefficients of binary features and standardized continuous ones?
  - One possibility: standardize by dividing by two standard deviations. Standard deviation then reduced to 0.5, same as for a symmetric Bernoulli random variable.

Cons

- Bad for comparing coefficients for the same predictors across different data sets (different studies); better keep the original scale.
- In general, interpretability often easier on "natural" scale.
- Standardizing binary features: lose the straightforward interpretability.

Preprocessing matters

The model is linear in the transformed inputs.

Examples

- `log(income)` has a different coefficient interpretation than `income`.
- Quantile transformations may help prediction but lose natural units.
- Scaling matters for regularized models: the penalty acts on coefficient sizes.
- Common logistic-regression software uses regularization by default, so inputs should be on roughly comparable scales.

Feature Importance: Z-score

Sampling distributions

- With further assumptions we can specify the sampling distribution of $\hat{\beta}$.
- We assume, for example, that $\varepsilon \sim N\left(0, \sigma^2\right)$ – a zero-mean Gaussian, meaning that we expect both negative and positive errors, most being within 2σ.
- We get that the sampling distribution is $\hat{\beta} \sim N\left(\beta, \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\sigma^2\right)$.
- Roughly, 95% confidence interval equals $\hat{\beta}_j \pm 2 \cdot \text{se}(\hat{\beta}_j)$, where $se\left(\hat{\beta}_j\right) = \hat{\sigma}\sqrt{v_j}$, where v_j is the j th diagonal element of $\left(\mathbf{X}^T\mathbf{X}\right)^{-1}$.
- Note that the confidence interval depends on the magnitude of the error, σ: the lower the error, the narrower the confidence interval.

Feature Importance: Z-score

Alternative measure of feature importance

Given by the absolute value of the standardized score,

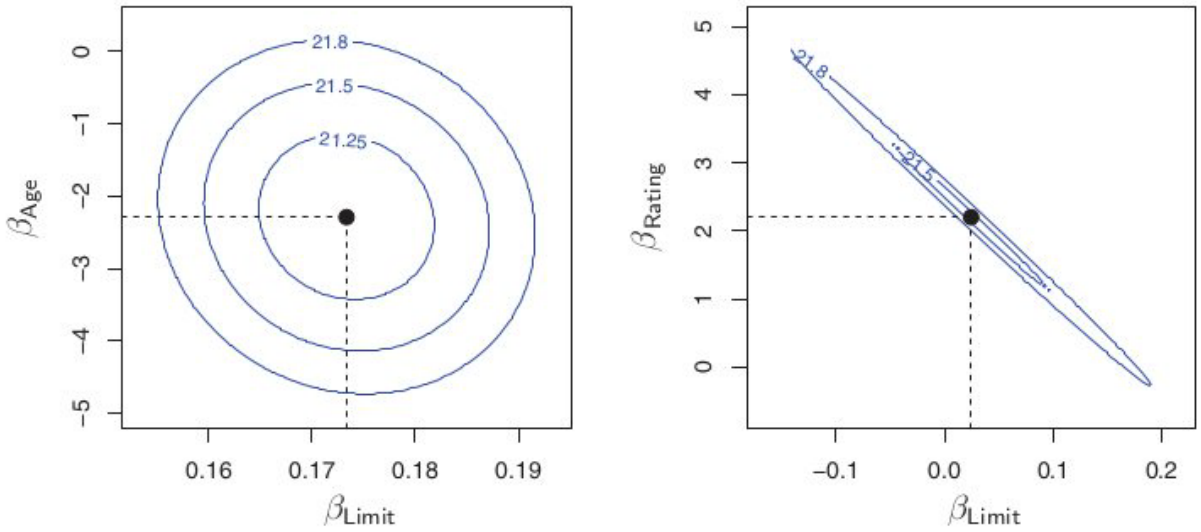
$$z_j = \frac{\hat{\beta}_j}{se\left(\hat{\beta}_j\right)}.$$

A standardized score of roughly 2 is significant at the 5% level.

- Account for error magnitude (residuals), correlation among features.
- Yet based on assumptions about the distribution of the error (which can often be checked, though).

Linear regression

Multicollinearity

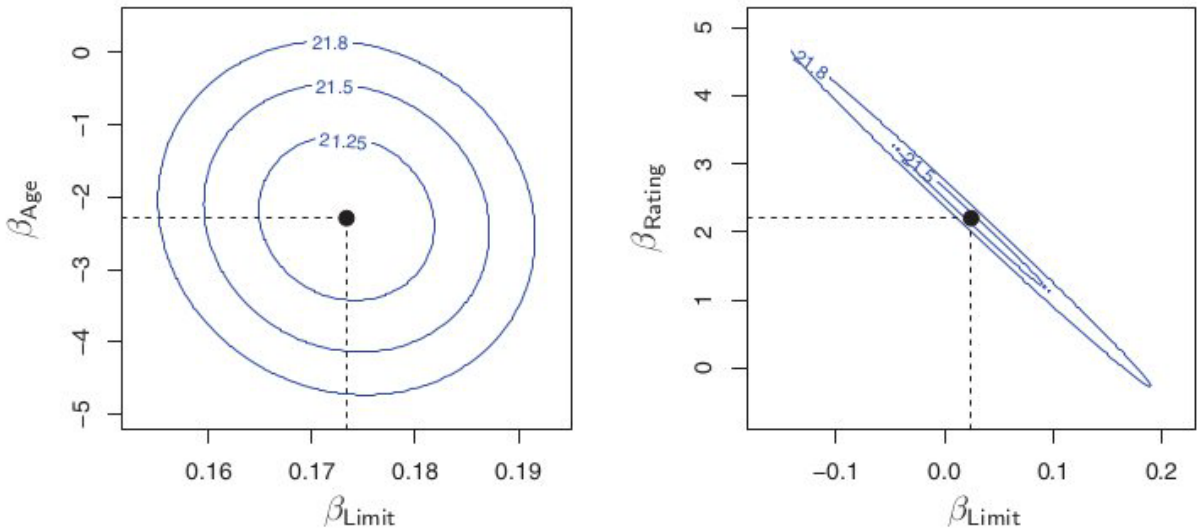


Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013).

- `lm(balance ~ limit + age)` versus `lm(balance ~ limit + rating)`.
- X-axis same scale: least squares estimates ± 4 std. errors.
- Left: True `limit` β between 0.15 and 0.20 with high confidence.

Linear regression

Multicollinearity



Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013).

Right: many `limit` βs result in equal values for RSS. A small change in data could cause the estimates to move far; the scale for limit is now from -0.2 to 0.2.

Linear regression

Multicollinearity

- Multicollinearity increases $se\left(\hat{\beta}_j\right)$, as it can be difficult to separate the individual effects of collinear variables on the response.
- This decreases the standardized score, reducing feature importance.
- Correlation is not only pairwise: a feature can be nearly explained by a combination of other features.

Multicollinearity

Instability under resampling

Truth: $y = x_1 + x_2 + \varepsilon$, with x_1 and x_2 highly correlated.

```
## selected
## x1 x2
## 15 15
```

- Different samples can select different representatives.
- This also affects feature importance in non-linear models: importance can be spread across correlated predictors.

Table of Contents

1 Representation

2 Learning

3 Interpretation

4 Sparsity

5 Generalizations

Sparsity

- For ease of interpretation, we might care about determining a smaller subset of features that exhibit the strongest effects.
- There are many ways to select features in a linear regression model.
- We can encourage sparsity by feature selection or shrinkage.

Shrinkage and interpretation

Ridge versus lasso

- Ridge (L_2) shrinks coefficients, usually keeping all predictors.
- Lasso (L_1) can set coefficients exactly to zero.
- With correlated predictors, lasso often picks one and drops the others.
- Elastic net mixes both penalties.

Caution

After feature selection or lasso, ordinary OLS confidence intervals and p-values do not have their usual interpretation.

Shrinkage

The lasso estimates, for regression, are given by

$$\hat{\beta}^{\text{lasso}} = \underset{\beta}{\text{argmin}} \left(\frac{1}{N} \sum_{i=1}^{n} \left(y^{(i)} - x_i^T \beta \right)^2 + \lambda \|\beta\|_1 \right),$$

where $\|\beta\|_1$ is the sum of absolute values of β, and λ controls the strength of shrinkage and can be tuned by cross-validation.

- The loss function balances the fit with coefficients' magnitude.
- Shrinks all coefficients by penalizing their absolute value.
- Unlike ridge regression, performs selection as some β go to zero.
- The tightest *convex relaxation* for penalizing with $|\beta\|_0$ –the number of non-zero coefficients; optimizing the latter is NP-hard.
- The higher λ, the stronger the penalty.

Lasso

```
library(glmnet)
```

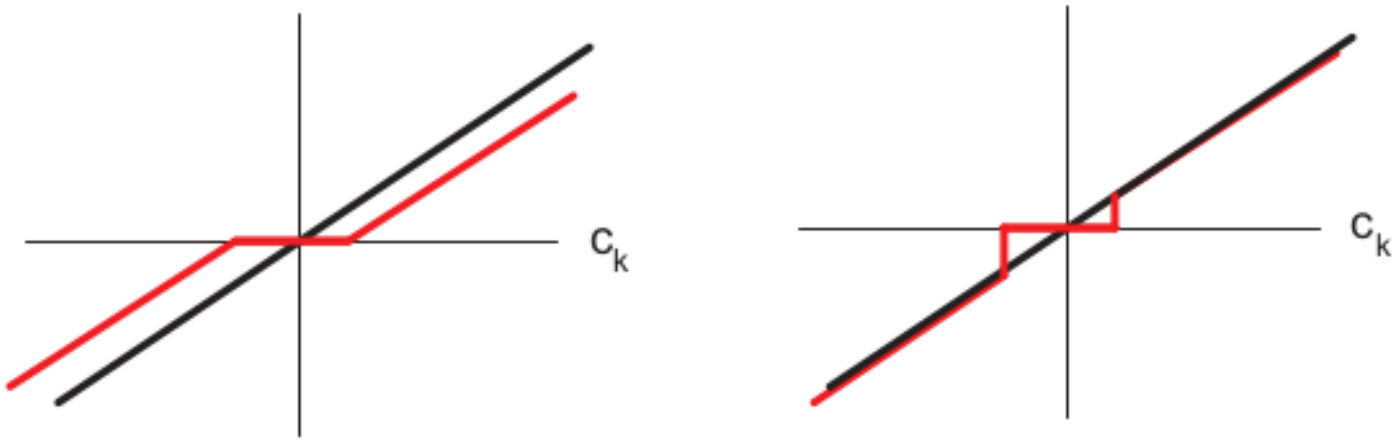
```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
fit <- glmnet(x =mtcars[, -1], y = mtcars[, 1], family = "gaussian"
              alpha = 1, lambda  = 2)
fit$beta
```

```
## 10 x 1 sparse Matrix of class "dgCMatrix"
##                s0
## cyl  -0.799971158
## disp  .
## hp   -0.002227335
## drat  .
## wt   -2.022363172
## qsec  .
## vs    .
## am    .
```

Lasso

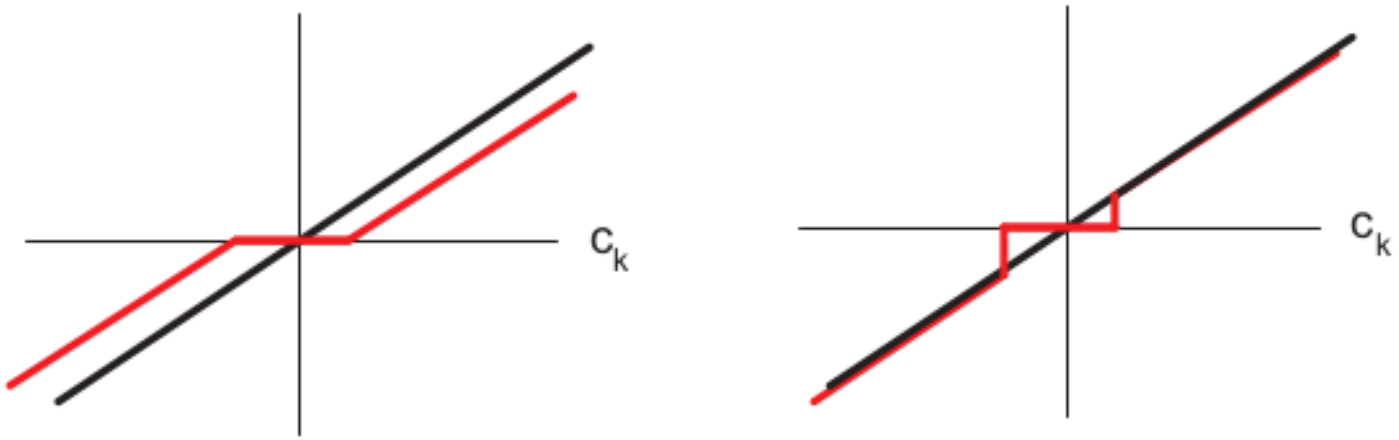


Soft (left) and hard (right) thresholding. The flat region is $[-\lambda, +\lambda]$. c_k proportional to correlation between X_k and the residual due to the other features. Dark line: least squares fit. Source: Murphy, 2012.

Hard thresholding

Sets β_j to 0 if $-\lambda \leq c_j \leq \lambda$, but it does not shrink β_j outside of this interval.

Lasso



Soft (left) and hard (right) thresholding. The flat region is $[-\lambda, +\lambda]$. c_k proportional to correlation between X_k and the residual due to the other features. Dark line: least squares fit. Source: Murphy, 2012.

Soft thresholding

- Unless $-\lambda \leq c_j \leq \lambda$, shifts the red line down (or up) by λ; it sets $\beta_j = 0$.
- Biased estimation: the line does not coincide with the diagonal, meaning that even large coefficients are shrunk.
- Undesirable: if the likelihood indicates (via c_j) that β_j should be large, we do not want to shrink it.

Debiasing

Simple workaround: relaxed lasso

Identify non-zero weights with lasso, re-estimate the chosen coefficients with least squares.

```
n <- 1000
x <- rnorm(n) ; z <- rnorm(n)
y <- x ^ 2 + z + rnorm(n)
fit <- glmnet(cbind(x, z), y, family = "gaussian",
              alpha = 1, lambda  = .8)
fit$beta
```

```
## 2 x 1 sparse Matrix of class "dgCMatrix"
##          s0
## x .
## z 0.1680217
```

```
lm(y ~ z)$coefficients
```

```
## (Intercept)           z
##   1.0251038   0.9467379
```

Debiasing

Relaxed lasso

- Use the lasso to identify non-zero predictors;
- fit least squares estimates using the non-zero predictors.

Alternatives

- Use the lasso to identify non-zero predictors;
- then apply the lasso again, only to the selected predictors;
- if using cross-validation to choose λ, the second time there is less competition from noise variables and a lower λ might do well.

Model selection

- It is common to use cross validation to pick a λ that gives good predictive accuracy.
- This is often not the same value as the one that is likely to recover the "true" model.
- To prevent relevant coefficients from being shrunk, cross validation will tend to pick a smaller value of λ.
- This will result in a less sparse model, containing irrelevant variables.

Correlation

- Lasso will tend to pick one among a group of correlated variables and set the β of the rest to 0.
- Can thus be unstable in the presence of correlated variables.

Note the lack of stability given correlations:

```
n <- 100
x <- rnorm(n)
z <- 0.5 * x + rnorm(x)
y <- x + z + rnorm(x)
fit <- glmnet(x =cbind(x, z), y = y, family = "gaussian",
              alpha = 1, lambda  = 1.6)
fit$beta
```

```
## 2 x 1 sparse Matrix of class "dgCMatrix"
##   s0
## x  0
## z
```

Bolasso

- Enforce a sparser estimator by thresholding an estimate of inclusion probabilities, e.g., at $t = 90\%$.
- Sample M bootstrap replicas and select features with lasso, for a fixed λ, on each.
- Select a variable that was chosen by lasso on at least t of sets.

Bolasso

```
library(bolasso)
bolasso_form <- bolasso(
  form = mpg ~ .,
  data = mtcars,
  n.boot = 20,
  implement = "glmnet"
)
```

```
selected_vars(bolasso_form, threshold = 1, lambda = "2")
```

```
## # A tibble: 1 x 2
##   variable  mean_coef
##   <chr>         <dbl>
## 1 Intercept      13.5
```

Lasso

Summary

- Lasso used extensively in high-dimensional model selection problems.
- The standard errors and p-values are no longer valid; `glmnet` does not even provide them.
- Can select at most N variables: a limitation in high-dimensional cases.
- An extension: elastic net.

Table of Contents

1 Representation

2 Learning

3 Interpretation

4 Sparsity

5 Generalizations

Generalized Linear Models

A generalization of both linear and logistic regression

$$g(\mathrm{E}(Y \mid x_1, x_2, \ldots, x_n)) = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n + \epsilon,$$

Three components:

- Link function g; e.g., identity and logit for linear and logistic regression, respectively.
- Probability distribution from the exponential family that defines $E_{Y|X_1,\ldots,X_n}$; e.g., Gaussian and Bernoulli.
- Weighted sum $\beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n$.

Non-linearities

- While attractively simple, the linear model is limited: true effects are rarely linear.
- We can extend linear models to handle non-linearities and interactions, usually sacrificing some interpretability.
- One way, which may retain much of the interpretability, is by discretizing continuous features.
- There exist continuous alternatives for introducing non-linearity, such as basis expansions, power transforms, etc.

Interactions

An interaction term is still linear in the coefficients:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2.$$

Interpretation

- The effect of x_1 now depends on x_2.
- The model is linear in the columns $(x_1, x_2, x_1 x_2)$.
- We gain expressiveness, but individual coefficients are less direct.

Discretize

What

- Partition continuous features into discrete values.
- Like constructing histograms, but focusing not on counting data points within a bin, but on the actual mapping of values to bins.
- For example, map interval $[0, 4.9]$ to category 'Suspenso'.
- Also known as quantization, binning and categorization.

Why?

- Discretize + one-hot: more expressive yet keep interpretability;
- for instance, discretizing can introduce nonlinearity to linear models.
- The learner assumes categorical or ordinal data.

Discretize + one-hot

Discretizing a continuous feature turns it into a categorical feature. After one-hot encoding, the bins are interpreted exactly like categories.

For a score $x \in [0, 10]$:

$$\begin{array}{ll} B_1 : 0 \leq x < 5 & \text{Suspenso, reference bin} \\ B_2 : 5 \leq x < 7 & \text{Aprobado} \\ B_3 : 7 \leq x < 9 & \text{Notable} \\ B_4 : 9 \leq x \leq 10 & \text{Sobresaliente} \end{array}$$

The linear model uses $K - 1$ dummy variables:

$$\hat{y} = \beta_0 + \beta_2 I(x \in B_2) + \beta_3 I(x \in B_3) + \beta_4 I(x \in B_4).$$

Discretized coefficient example

Fitting `lm(y ~ bin)` with `B1` as the reference bin:

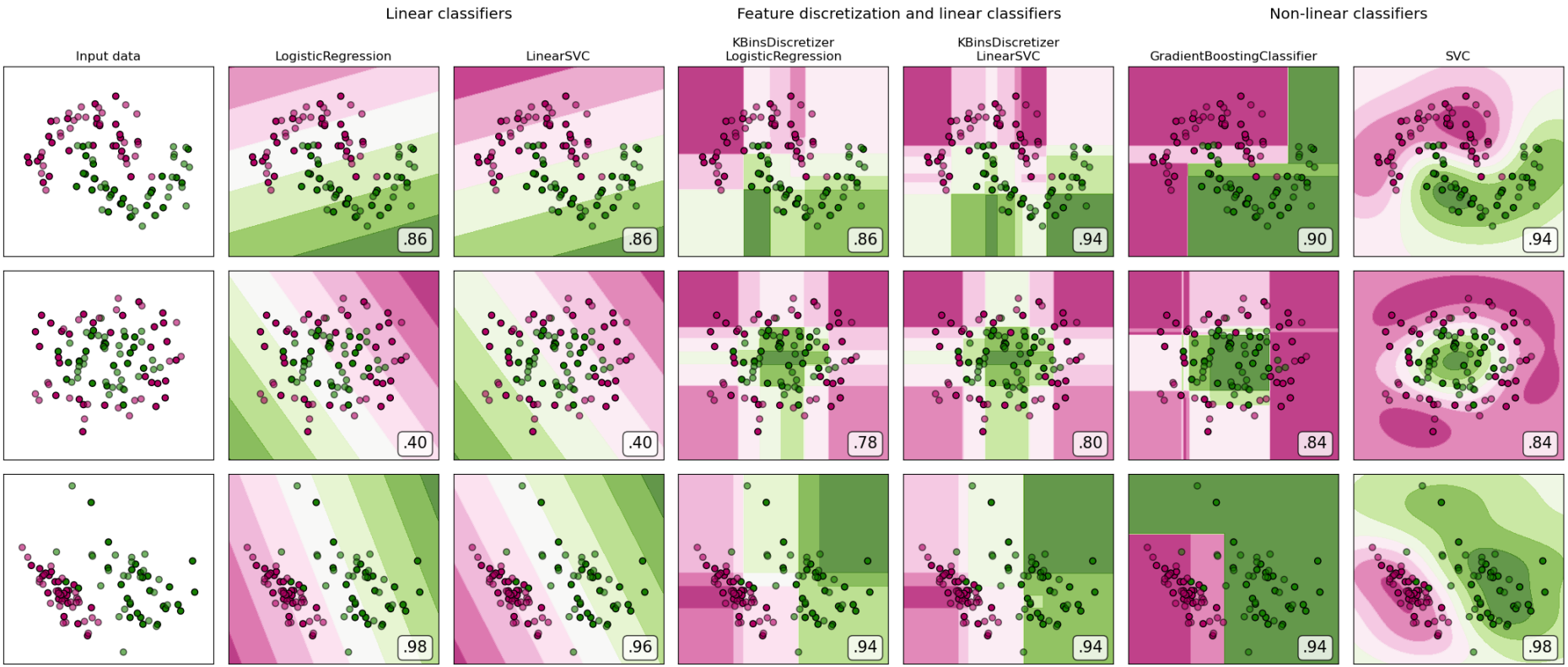
coefficient	estimate
`(Intercept)`	53.7
`binB2`	8.3
`binB3`	19.3
`binB4`	29.3

Interpreting the coefficients

Interpretation

- The intercept is the prediction for the reference bin: `B1` (`Suspenso`).
- `binB2` is the change from `B1` (`Suspenso`) to `B2` (`Aprobado`).
- `binB3 - binB2` is the change from `B2` (`Aprobado`) to `B3` (`Notable`).
- In logistic regression, the same coefficients are interpreted on the log-odds scale.

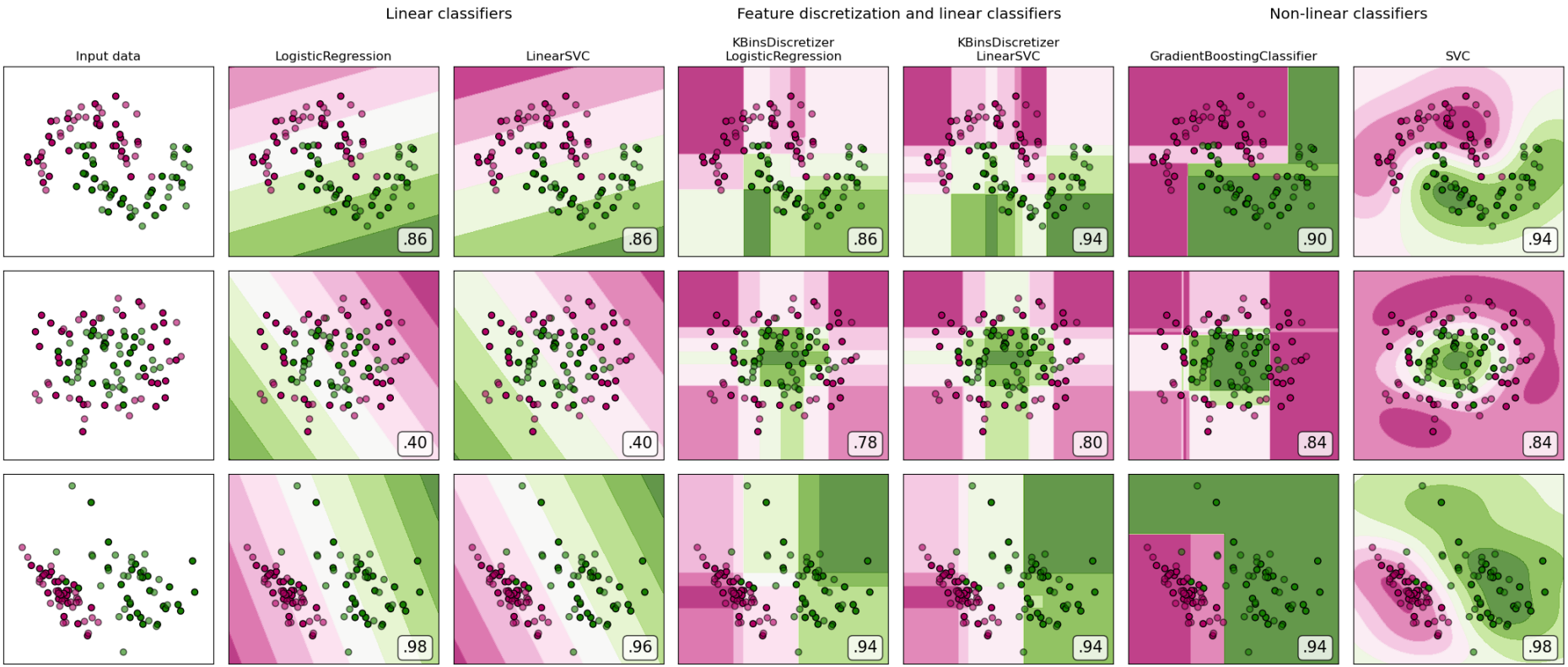
Non-linearity



Source: scikit-learn: Feature discretization

Training: solid; testing: semi-transparent; lower right: test set accuracy.

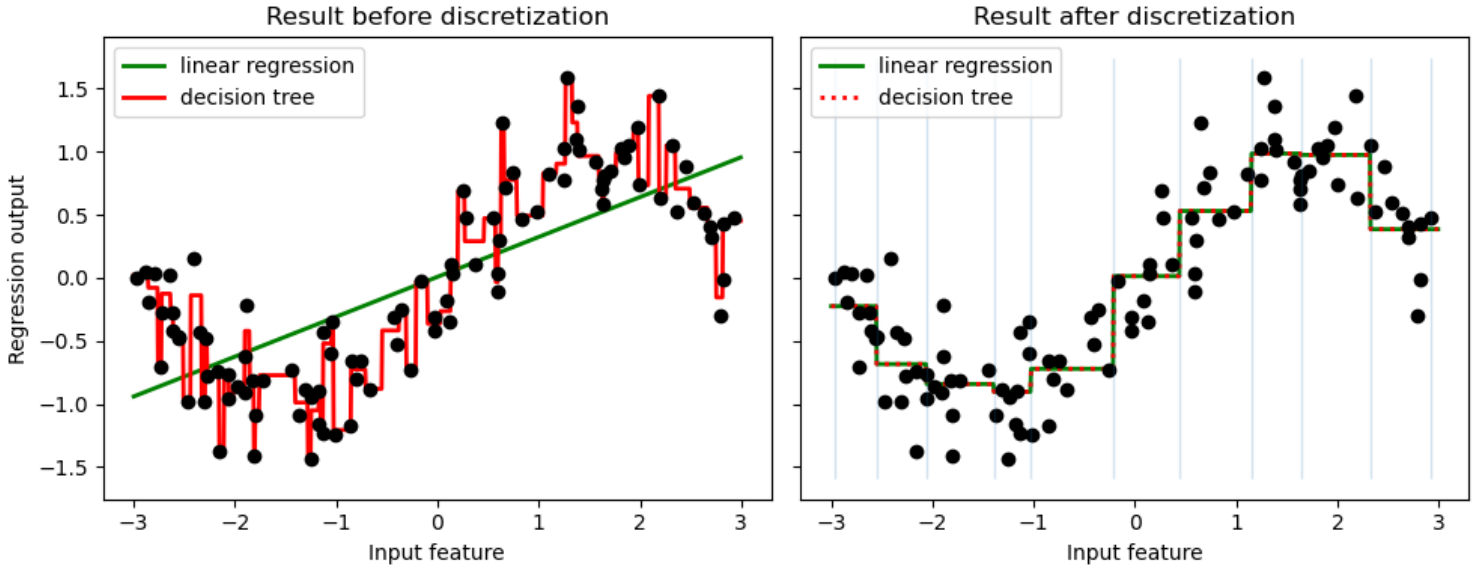
Non-linearity



Source: scikit-learn: Feature discretization

- Linearly non-separable: improves performance.
- Linearly separable: slightly degrades performance.
- Note: in high-dimensional spaces, we can usually separate data linearly.

Non-linearity



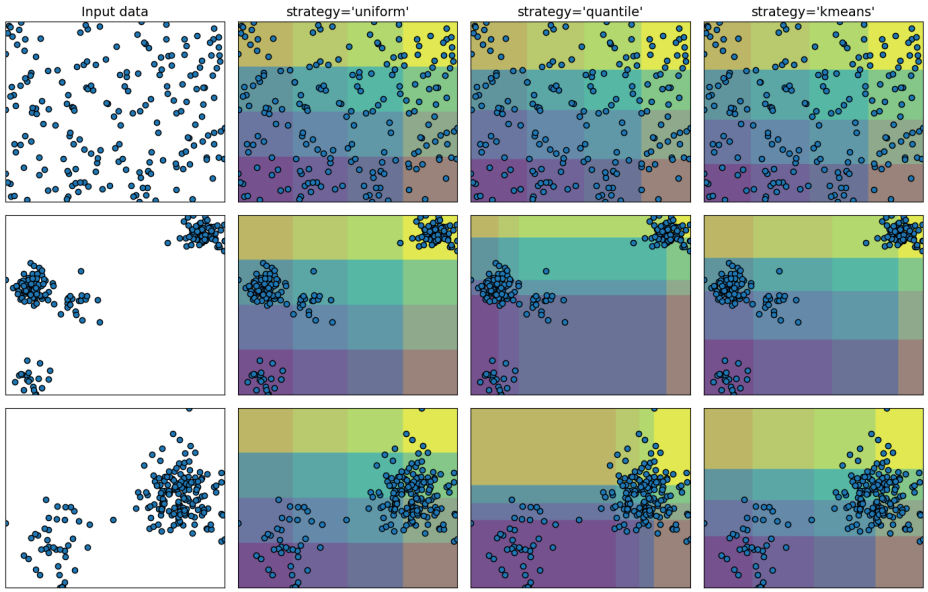
Source: scikit-learn: Using KBinsDiscretizer to discretize continuous features

- Linear regression improves with discretization while regression tree degrades.
- Tree and linear regression identical after discretization.

KBinsDiscretizer

Strategy

- `uniform`: constant-width bins.
- `quantile`: equally populated bins (default).
- `kmeans`: k-means clustering.



Remarks

Drawback

- Adds complexity: Let $n = 100$ with $K = 3$ for all variables; how many variables with discretization + one-hot?
- Lose information: lose properties of numbers; e.g., 4.8 and 5.0 are 'Suspenso' and 'Aprobado', respectively, just like 0 and 6.8.
- Discontinuities might be very unrealistic for some phenomena.
- Assumes that the relationship between the feature and the target is flat within an interval; this is often less reasonable than assuming linearity.

Over/under-fitting

- The more bins, the better the fit;
- however, worse estimators because there are fewer points in each bin.
- Avoid tuning parameters on test data; use defaults or validation set (or cross-validation).